

- ***Page 1: Introduction to Graph Theory***

What is a Graph?.

A graph is a mathematical structure used to model pairwise relations between objects. It consists of a set of ***vertices*** (also called nodes or points) and a set of ***edges*** connecting pairs of vertices. Graphs are incredibly versatile and are used to represent a wide variety of relationships, from social networks to road maps to computer networks...

- ****Vertices:**** Represent the objects or entities in the system. For example, in a social network, vertices could represent individuals.
- ****Edges:**** Represent the relationships between the objects. In a social network, an edge might represent a friendship between two individuals. Edges can be ***directed*** (indicating a one-way relationship) or ***undirected*** (indicating a two-way relationship).

Types of Graphs.

- ***There are many types of graphs, each with its own properties. Some common types include:****
- ****Undirected Graph:**** Edges have no direction. If there's an edge between vertices A and B, it means A is connected to B and B is connected to A.
- ****Directed Graph (Digraph):**** Edges have a direction. An edge from A to B means there's a connection from A to B, but not necessarily from B to A.
- ****Weighted Graph:**** Edges have associated weights, representing a value or cost associated with the connection. This is common in transportation networks where weights might represent distances or travel times.
- ****Simple Graph:**** A graph with no loops (edges connecting a vertex to itself) and no multiple edges between the same pair of vertices.
- ****Complete Graph:**** A simple graph where every pair of vertices is connected by a unique edge.

- *Page 2: Graph Representations*

Adjacency Matrix.

An adjacency matrix is a square matrix where the element at row i and column j represents the connection between vertex i and vertex j ...

- **Undirected Graph:** A value of 1 indicates an edge, 0 indicates no edge. The matrix is symmetric.
- **Directed Graph:** A value of 1 indicates an edge from vertex i to vertex j . The matrix is not necessarily symmetric.
- **Weighted Graph:** The matrix elements contain the weight of the edge. 0 or a special value (like infinity) indicates no edge.
- **Example:** Consider an undirected graph with vertices A, B, C. If A is connected to B and C, and B is connected to C, the adjacency matrix would be:

```

  A B C
A 0 1 1
B 1 0 1
C 1 1 0

```

Adjacency List.

An adjacency list represents the graph as a list of vertices, where each vertex has a list of its neighbors...

- **Efficient for sparse graphs:** Graphs with relatively few edges compared to the number of vertices.
- **Space efficient:** Only stores the existing edges.
- **Example:** For the same graph above, the adjacency list would be:
- A: [B, C]

- B: [A, C]
- C: [A, B]
- *Page 3: Graph Traversal Algorithms**

Breadth-First Search (BFS).

BFS explores a graph level by level It starts at a source vertex and visits all its neighbors before moving to their neighbors It uses a queue data structure...

- **Applications:** Finding shortest paths in unweighted graphs, social network analysis.
- **Algorithm:**

Depth-First Search (DFS).

DFS explores a graph by going as deep as possible along each branch before backtracking It uses a stack data structure (implicitly through recursion)...

- **Applications:** Detecting cycles, topological sorting, finding connected components.
- **Algorithm:**

- *Page 4: Minimum Spanning Trees**

A minimum spanning tree (MST) is a tree that connects all vertices of a weighted graph with the minimum possible total edge weight There are several algorithms to find MSTs...

Prim's Algorithm.

Prim's algorithm starts with a single vertex and iteratively adds the edge with the minimum weight that connects a vertex in the tree to a vertex outside the tree...

- **Greedy algorithm:** Makes locally optimal choices at each step.
- **Uses a priority queue:** To efficiently find the minimum-weight edge.

Kruskal's Algorithm.

Kruskal's algorithm sorts the edges by weight and iteratively adds edges to the tree, as long as adding the edge does not create a cycle...

- **Uses a disjoint-set data structure:** To efficiently check for cycles.
- **Page 5: Shortest Path Algorithms**

Dijkstra's Algorithm.

Dijkstra's algorithm finds the shortest paths from a single source vertex to all other vertices in a weighted graph with non-negative edge weights...

- **Greedy algorithm:** Similar to Prim's, it iteratively selects the vertex with the smallest distance from the source.
- **Uses a priority queue:** To efficiently find the vertex with the minimum distance.

Bellman-Ford Algorithm.

Bellman-Ford algorithm finds the shortest paths from a single source vertex to all other vertices in a weighted graph, even if the graph contains negative edge weights (but not negative cycles)...

- **Handles negative edge weights:** Unlike Dijkstra's.
- **Dynamic programming approach:** Iteratively relaxes edges until no further improvements can be made. Detects negative cycles.

These notes provide a foundational understanding of graph theory. Further exploration would involve studying more advanced topics like network flow, matching, and planarity...